

LifeOS

LifeOS — Database Schema & API Design

Version: 1.0 · **Date:** 2026-11-08 **Companion docs:** `lifeos-features-spec.md` (what to build) · `lifeos-ui-flow.md` (screens) **Stack:** Flutter (Drift/SQLite local) ⇄ Supabase (Postgres + Auth + Edge Functions) · Cloudflare R2 (encrypted blobs) · FCM (push)

---

1. Architecture Principles

1. **Offline-first:** the phone's SQLite is the working database. The server is backup, sync, and the ATC cron brain. Every feature works with zero connectivity.
2. **Local and cloud schemas are the same shape.** One logical model, two engines (Drift mirrors Postgres). Sync is dumb and reliable, not clever.
3. **IDs are client-generated UUIDv7** (time-ordered → index-friendly, no server round-trip to create rows offline).
4. **Soft delete everywhere** (`deleted_at`), 30-day trash, hard-purged by cron.
5. **Every syncable row carries sync metadata** (see §4).
6. **Documents are blobs + metadata.** Postgres never stores file bytes; R2 stores ciphertext; the `documents` row stores metadata + extracted text for search.
7. **Zero-knowledge vault:** files are AES-256-GCM encrypted on device with keys derived from the user's master key. The server stores ciphertext it cannot read.

---

2. Entity Relationship Overview

```
users 1—n households n—n users (household_members)
users 1—n persons (profiles: Me, spouse, kids, parents)
users 1—n folders
persons 1—n documents n—1 folders
documents 1—n document_files (pages/versions)
documents 1—n document_shares (safe links)
documents 1—n extracted_fields
users 1—n atc_rules 1—n tasks (source link)
tasks n—1 persons · n—1 domains · 1—n task_reminders · 1—n subtasks
tasks 1—1 payments (payment-type tasks)
payments 1—n expenses (completion writes an expense)
users 1—n expenses n—1 categories
users 1—n score_snapshots (daily)
users 1—n devices (push tokens, sync cursors)
users 1—n user_settings (theme template, locks, quiet hours)
```

---

3. Schema (Postgres DDL — Drift mirrors the same columns)

> Types shown as Postgres; Drift equivalents: `uuid→TEXT`, `timestamptz→INTEGER (epoch ms, always UTC)`, `jsonb→TEXT(JSON)`, `numeric→INTEGER (paisa/cents!)`. **Money rule: store minor units as BIGINT (`amount_minor`), never floats.**

3.1 Identity & settings

```
-- Supabase auth.users provides id/email/phone. Public profile:
create table users (
  id            uuid primary key references auth.users(id),
  display_name  text not null,
  role_profile  text check (role_profile in ('student','professional','parent','business','other')),
  locale        text not null default 'en',        -- 'en' | 'ur'
  timezone      text not null default 'Asia/Karachi',
  salary_day    smallint,                          -- 1..31, nullable (salary-aware mode)
  premium_until timestamptz,
  created_at    timestamptz not null default now()
);

create table user_settings (
  user_id         uuid primary key references users(id),
  theme_template  text not null default 'flagship',  -- 'flagship'|'funky'|'classic'
  dark_mode       text not null default 'system',
  text_scale      real not null default 1.0,
  app_lock        boolean not null default false,
  vault_lock      boolean not null default true,
  auto_lock_min   smallint not null default 5,
  quiet_start     time not null default '22:00',
  quiet_end       time not null default '07:00',
  digest_time     time not null default '07:30',
  notif_domains   jsonb not null default '{}',       -- {"money":true,"health":false,...}
  backup_mode     text not null default 'wifi',      -- 'off'|'wifi'|'always'
  updated_at      timestamptz not null default now()
);

create table devices (
  id            uuid primary key,
  user_id       uuid not null references users(id),
  platform      text not null,                       -- 'android'|'ios'
  fcm_token     text,
  last_synced_at timestamptz,                        -- server cursor per device
  created_at    timestamptz not null default now()
);

create table persons (                               -- Me / family profiles
  id          uuid primary key,
  user_id     uuid not null references users(id),
  name        text not null,
  relation    text not null default 'self',          -- self|spouse|child|parent|other
```

```
  avatar      text,                                  -- emoji or asset key
  dob         date,
  blood_group text,                                  -- Emergency Card (P1)
  sort_order  smallint not null default 0,
  -- sync meta (§4) on every syncable table:
  updated_at  timestamptz not null default now(),
  deleted_at  timestamptz,
  version     bigint not null default 1
);
```

3.2 Vault

```
create table folders (
  id         uuid primary key,
  user_id    uuid not null references users(id),
  name       text not null,
  icon       text not null default '📁',
  kind       text not null default 'custom',
  -- system kinds: identity|education|medical|bills|property|vehicles|insurance|work
  sort_order smallint not null default 0,
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

create table documents (
  id              uuid primary key,
  user_id         uuid not null references users(id),
  person_id       uuid references persons(id),
  folder_id       uuid references folders(id),
  title           text not null,
  doc_type        text not null default 'other',
  -- cnic|passport|visa|license|bill|fee_voucher|receipt|prescription|report|
  -- degree|certificate|warranty|policy|agreement|other
  issuer          text,                    -- 'K-Electric', school name…
  ref_number      text,                    -- account / roll / policy no.
  issue_date      date,
  expiry_date     date,                    -- drives Expiry Guard
  expiry_alerts   jsonb,                   -- override ladder e.g. [180,90,30,7]
  amount_minor    bigint,                  -- bills/fees: amount in paisa
  currency        text not null default 'PKR',
  due_date        date,                    -- bills/fees
  period_month    date,                    -- billing month (first of month)
  ocr_text        text,                    -- extracted full text (search)
  classification_confidence real,
  favorite        boolean not null default false,
  tags            text[] not null default '{}',
  source          text not null default 'scan',   -- scan|import|share_in
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

-- Full-text search (server). Local: SQLite FTS5 over title+ocr_text+issuer.
alter table documents add column fts tsvector
  generated always as (to_tsvector('simple',
```

```
    coalesce(title,'')||' '||coalesce(issuer,'')||' '||coalesce(ocr_text,''))) stored;
create index documents_fts on documents using gin(fts);
create index documents_expiry on documents(user_id, expiry_date)
  where deleted_at is null and expiry_date is not null;
create index documents_person on documents(user_id, person_id, doc_type);

create table document_files (               -- pages / re-scans
  id             uuid primary key,
  document_id    uuid not null references documents(id),
  user_id        uuid not null references users(id),
  page_no        smallint not null default 1,
  r2_key         text not null,             -- 'u/{user}/d/{doc}/{file}.bin'
  byte_size      bigint not null,
  mime           text not null default 'application/pdf',
  sha256         text not null,             -- of ciphertext (integrity)
  enc_iv         text not null,             -- AES-GCM nonce (b64)
  enc_key_wrapped text not null,            -- file key wrapped by user master key
  thumb_r2_key   text,                      -- encrypted thumbnail
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

create table extracted_fields (             -- structured OCR output
  id          uuid primary key,
  document_id uuid not null references documents(id),
  user_id     uuid not null references users(id),
  field       text not null,                -- amount|due_date|expiry|name|number…
  value       text not null,
  confidence  real,
  corrected   boolean not null default false, -- user fixed it → feeds learning
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

create table document_shares (              -- safe links
  id           uuid primary key,
  document_id  uuid not null references documents(id),
  user_id      uuid not null references users(id),
  token        text not null unique,        -- random 128-bit, url-safe
  expires_at   timestamptz not null,
  pin_hash     text,
  max_views    smallint,
  view_count   smallint not null default 0,
  revoked_at   timestamptz,
  created_at   timestamptz not null default now()
```

```
);
-- Note: shared files are re-encrypted with a share key embedded in the URL
-- fragment (#k=...), so the server still never holds a readable key.
```

3.3 Tasks & ATC

```
create table tasks (
  id            uuid primary key,
  user_id       uuid not null references users(id),
  person_id     uuid references persons(id),
  domain        text not null default 'mind',
  -- mind|money|health|people|work|soul|vault
  title         text not null,
  notes         text,
  due_at        timestamptz,
  all_day       boolean not null default true,
  priority      smallint not null default 1,          -- 0 low, 1 med, 2 high
  duration_min  smallint not null default 30,         -- Overload Shield
  status        text not null default 'open',         -- open|done|skipped
  completed_at  timestamptz,
  repeat_rule   text,                                  -- RRULE string, nullable
  atc_rule_id   uuid references atc_rules(id),         -- ⚡ provenance
  source_document_id uuid references documents(id),    -- "Created from: bill…"
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);
create index tasks_today on tasks(user_id, status, due_at) where deleted_at is null;

create table subtasks (
  id uuid primary key, task_id uuid not null references tasks(id),
  user_id uuid not null references users(id),
  title text not null, done boolean not null default false, sort_order smallint not null default 0,
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

create table task_reminders (
  id uuid primary key, task_id uuid not null references tasks(id),
  user_id uuid not null references users(id),
  fire_at timestamptz not null,
  fired boolean not null default false,       -- local only in practice
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

create table atc_rules (
  id            uuid primary key,
  user_id       uuid not null references users(id),
```

```
  person_id     uuid references persons(id),
  kind          text not null,       -- 'fixed'|'bill_pattern'|'fee'|'appointment_followup'
  name          text not null,       -- 'Electricity — K-Electric'
  issuer        text,                -- matches documents.issuer for pattern rules
  amount_minor  bigint,              -- fixed amount; null = varies (OCR fills it)
  amount_avg_minor bigint,           -- learned average (bill_pattern)
  due_day       smallint,            -- fixed: day of month
  expected_window jsonb,             -- bill_pattern: {"from":10,"to":14}
  cycle         text not null default 'monthly',  -- monthly|quarterly|semester|yearly|once
  skip_months   smallint[] not null default '{}', -- e.g. {6,7} school vacation
  installment_total smallint,        -- committee: 24
  installment_done  smallint,        -- 7
  remind_days_before smallint not null default 5,
  status        text not null default 'proposed', -- proposed|active|paused|ended
  confidence    real,                -- pattern detection confidence
  last_generated_period date,        -- idempotency: one task per period
  stats         jsonb not null default '{}',      -- {"created":14,"on_time":12}
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);
```

3.4 Money

```
create table categories (
  id uuid primary key, user_id uuid not null references users(id),
  name text not null, icon text, kind text not null default 'expense',
  system_key text,                    -- rent|fees|utilities|internet|installment…
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);

create table payments (               -- one row per expected payment occurrence
  id            uuid primary key,
  user_id       uuid not null references users(id),
  task_id       uuid references tasks(id),
  atc_rule_id   uuid references atc_rules(id),
  category_id   uuid references categories(id),
  title         text not null,
  amount_minor  bigint,               -- null until bill OCR fills it
  currency      text not null default 'PKR',
  due_date      date not null,
  period_month  date not null,        -- forecast grouping
  status        text not null default 'upcoming', -- upcoming|paid|overdue|skipped
  paid_at       timestamptz,
  receipt_document_id uuid references documents(id),
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);
create index payments_forecast on payments(user_id, period_month, status);

create table expenses (
  id uuid primary key, user_id uuid not null references users(id),
  category_id uuid references categories(id),
  payment_id  uuid references payments(id),   -- auto-logged from completion chain
  amount_minor bigint not null, currency text not null default 'PKR',
  spent_on date not null, note text,
  receipt_document_id uuid references documents(id),
  updated_at timestamptz not null default now(), deleted_at timestamptz, version bigint not null
default 1
);
create index expenses_month on expenses(user_id, spent_on);
```

3.5 Score & engagement

```
create table score_snapshots (
  user_id    uuid not null references users(id),
  day        date not null,
  score      smallint not null,
  components jsonb not null,  -- {"tasks":80,"payments":70,"documents":60,...}
  drags      jsonb not null,  -- [{"kind":"overdue_payment","id":"…","points":-6}]
  primary key (user_id, day)
);
```

3.6 Households (P2 — create tables now, ship UI later)

```
create table households (
  id uuid primary key, owner_id uuid not null references users(id),
  name text not null, created_at timestamptz not null default now()
);
create table household_members (
  household_id uuid references households(id), user_id uuid references users(id),
  role text not null default 'member',   -- admin|member|viewer
  primary key (household_id, user_id)
);
-- Sharing grain: folder level and task level.
alter table folders add column household_id uuid references households(id);
alter table tasks   add column assigned_to uuid references users(id);
```

---

4. Sync Protocol (offline-first)

4.1 Metadata on every syncable table

`updated_at` (server-set on write) · `deleted_at` · `version` (optimistic counter). Client keeps an **outbox**: `pending_ops(id, table, row_id, op, payload, base_version, queued_at)`.

4.2 Flow (per device)

```
PUSH: client sends outbox batch (≤500 ops)
  → server applies in order; for each op:
      row.version == base_version  → apply, version++, updated_at=now()
      else → CONFLICT, return server row
PULL: client sends last_synced_at cursor
  → server returns rows where updated_at > cursor (≤1000/page, ordered)
  → client upserts, advances cursor (devices.last_synced_at)
```

4.3 Conflict policy (simple, explainable)

- Default: **last-write-wins per row** (server `updated_at` decides).
- Exceptions:
- `tasks.status='done'` **always wins** over a concurrent edit (never un-complete).
- `payments.paid_at` set wins over any other change.
- `documents` file content never conflicts (files are immutable; a re-scan adds a new `document_files` row).
- Conflicted losing payloads land in local `conflict_log` for a quiet "restored an older edit" undo (P1).

4.4 Blob sync

- Upload: client asks `POST /files/presign` → PUT ciphertext straight to R2 → confirm row.
- Download: lazy; thumbnails first, full file on open. Local LRU cache (512 MB default, setting).
- Backup mode off → documents stay local-only; `document_files.r2_key` null until enabled.

---

5. Security Model

5.1 Key hierarchy (zero-knowledge vault)

```
Master Key MK (256-bit, random, created on device)
 ├─ wrapped by: device keystore (StrongBox/Secure Enclave)   → daily use
 ├─ wrapped by: KDF(recovery phrase, Argon2id)               → account recovery
 └─ wraps: per-file keys FK (one per document_files row)
File bytes = AES-256-GCM(FK, file)   — encrypted BEFORE upload
```

- Lose phone + recovery phrase = files unrecoverable. Stated clearly at backup opt-in. This is the trust feature — own it.
- `ocr_text` **is** stored server-side plaintext for search (explicit toggle: "Private search mode" keeps OCR text local-only; server search then unavailable).

5.2 Row-Level Security (Supabase)

```
alter table documents enable row level security;
create policy own_rows on documents
  using (user_id = auth.uid())
  with check (user_id = auth.uid());
-- Same policy on every user table. Household tables add:
--   or exists(select 1 from household_members hm
--             where hm.household_id = folders.household_id
--               and hm.user_id = auth.uid())
```

5.3 API hard rules

- JWT (Supabase auth) on everything except `GET /share/:token`.
- Share links: token 128-bit random; PIN attempts rate-limited (5/hour/IP); view counting server-side.
- Free-tier quota enforced server-side: `max 25 documents`, `max 3 active atc_rules` (premium claim in JWT).

---

6. API Design

Most CRUD = **Supabase PostgREST + RLS directly** (fast to ship, typed client). Custom logic = **Edge Functions**. Listed as REST for clarity:

6.1 Sync

Method	Path	Notes
POST	`/sync/push`	`{deviceId, ops:[…]}` → `{applied:[], conflicts:[]}`
GET	`/sync/pull?cursor=…&limit=1000`	changed rows, all tables, paged

6.2 Vault & files

Method	Path	Notes
POST	`/files/presign`	`{docId, pages, bytes, sha256}` → R2 presigned PUTs; enforces quota
POST	`/files/confirm`	marks upload complete
POST	`/documents/:id/classify`	**premium**: cloud LLM classify/extract when on-device confidence <0.7. Body: OCR text only (never the image). → `{doc_type, fields[], confidence}`
POST	`/documents/:id/share`	`{ttl, pin?, maxViews?}` → `{url}` (key in fragment)
DELETE	`/shares/:id`	revoke
GET	`/share/:token`	public; serves ciphertext + metadata; counts view

6.3 ATC engine (Edge Functions + pg_cron)

Method	Path	Notes
POST	`/atc/evaluate`	called after each document save & payment completion; runs pattern detection → may return a **proposal**
POST	`/atc/rules/:id/respond`	`{accept:bool}` — proposal → active/dismissed
POST	`/atc/generate`	idempotent per `(rule, period)`; also run by cron daily 00:10 local-tz batches; creates tasks+payments for the coming window
GET	`/forecast?month=2026-11`	`{total_minor, items:[…], salary_aware:{…}}`

Pattern detection (bill_pattern), server-side, boring and reliable:

```
group documents by (user, issuer) where doc_type='bill'
if ≥2 consecutive months AND stddev(arrival_day) ≤ 4:
    propose rule {expected_window: μ±σ, amount_avg}
confidence = f(cycles_observed, regularity); propose at ≥0.7
amount spike flag: new bill > 2× rolling_avg → notify "Unusually high ⚠"
```

6.4 Score (computed, never stored as truth until snapshot)

Method	Path	Notes
GET	`/score/today`	computes from live data; weights per spec §9; unused domains redistribute
GET	`/score/history?days=30`	from `score_snapshots`
cron	daily 23:59 user-tz	writes `score_snapshots`, builds morning digest payload

The client computes the same score locally when offline — same formula, shared constants file (`score_weights.json` shipped in-app, version-pinned).

6.5 Notifications

Trigger	Mechanism
Task/payment reminders	**local notifications** (scheduled from `task_reminders`) — offline-safe
Morning digest	FCM push at `digest_time` (server composes; falls back to local if token stale)
ATC proposal, family events	FCM data message → in-app card
Expiry Guard ladder	generated into `task_reminders` locally when `expiry_date` set/changed

6.6 Account & data

Method	Path	Notes
GET	`/export`	async job → ZIP (PDFs as stored ciphertext + key manifest decryptable in-app, data as CSV/JSON) → signed URL, 24h
DELETE	`/account`	queues purge (30 days), cancels subscription state

---

7. Scale Notes (100k → 1M users)

Concern	Answer
Doc search at 247+ docs/user	Local FTS5 answers instantly; server GIN index only for multi-device restore
ATC cron fan-out	partition by timezone bucket; generate is idempotent (`last_generated_period`) so retries are safe
R2 storage cost	~5 MB/user median → 1M users ≈ 5 TB ≈ $75/mo storage, **$0 egress** (why R2 over S3)
Hot tables	`tasks` , `payments` — covered composite indexes above; archive `status='done'` older than 18 months to cold partition
Sync amplification	per-device cursor prevents re-pulls; ops batched; payload deltas only
Postgres at 1M	single primary + read replica is fine at this shape; nothing here needs microservices

---

8. Drift (local) divergences — the only ones allowed

1. `fts` column → SQLite **FTS5 virtual table** `documents_fts(title, issuer, ocr_text)` kept in sync by triggers.
2. `task_reminders.fired` exists only locally (server doesn't care).
3. Local-only tables: `pending_ops` (outbox), `conflict_log` , `blob_cache(r2_key, local_path, last_used)` .
4. Timestamps as epoch-ms integers, always UTC; convert at the edge of the UI.

---

9. Build Checklist for the Dev Team

- [ ] Migration 001: §3.1–3.5 tables + RLS policies + indexes
- [ ] Drift schema generated from the same model file (single source: `schema.dbml` → codegen both ways)
- [ ] Sync endpoints + outbox client (test: airplane-mode week, then sync)
- [ ] Presign/confirm upload path with quota enforcement
- [ ] ATC evaluate/generate functions + pg_cron jobs (idempotency test: run generate twice, expect 0 duplicates)
- [ ] Score function shared constants + daily snapshot cron
- [ ] Share links: token issue, PIN limit, revoke, expiry sweep cron
- [ ] Export + delete-account jobs (compliance)

End — LifeOS data model & API v1.0.
